

Comunicación Serial Básica Arduino

¿Qué es?

La comunicación serial es un método para transmitir datos entre el Arduino y el ordenador (o entre Arduino y otros dispositivos) a través del puerto USB. Permite que tu microcontrolador "hable" con el PC, enviando información que puedes ver en una consola. Arduino utiliza el protocolo **UART (Universal Asynchronous Receiver/Transmitter)** para esta comunicación.

¿Para qué sirve?

- Enviar mensajes desde Arduino al PC para ver el estado del programa
- Depurar tu código viendo qué sucede en cada momento
- Recibir instrucciones desde el PC hacia Arduino
- Monitorear sensores y valores en tiempo real

Baudios

Los baudios representan la velocidad a la que se transmiten los datos. Es como la velocidad de comunicación entre Arduino y tu PC. Los valores más comunes son:

- 9600: El más común, usado en la mayoría de proyectos básicos
- 115200: Más rápido, usado en proyectos más avanzados
- 19200, 38400, 57600: Velocidades intermedias

Para que Arduino y tu PC se entiendan, ambos deben usar la misma velocidad de comunicación.

Inicialización de la comunicación Serial

Para comenzar a usar la comunicación serial, debes inicializarla en la función `setup()` usando `Serial.begin()`.

```
Serial.begin(baudrate);
```

Ejemplo:

```
void setup() {  
  Serial.begin(9600); // Inicializa comunicación serial a 9600 baudios  
}
```

Enviar datos al PC desde el microcontrolador

Existen dos funciones principales para enviar datos:

`Serial.print()` : Envía datos a través del puerto serial **sin agregar un salto de línea** al final. Útil cuando quieres controlar exactamente cómo se formatean los datos.

Sintaxis:

```
Serial.print(data);
```

Ejemplo:

```
Serial.print("Hola");  
Serial.print(" ");  
Serial.print("Mundo");
```

// Resultado en consola: Hola Mundo

`Serial.println()`: Envía datos a través del puerto serial **y agrega un salto de línea** al final. Es más cómodo para cuando quieres cada dato en una línea diferente.

Sintaxis:

```
Serial.println(data);
```

Ejemplo:

```
Serial.println("Línea 1");
Serial.println("Línea 2");
Serial.println("Línea 3");
// Resultado en consola:
// Línea 1
// Línea 2
// Línea 3
```\n`
```

## ## Monitor Serie del IDE Arduino

El **Monitor Serie** es la herramienta que utiliza para ver los mensajes que envía tu Arduino.

**\*\*Cómo abrir el Monitor Serie:\*\***

1. En el IDE de Arduino, ve a: `Herramientas → Monitor Serie`
2. O usa el atajo: `Ctrl + Shift + M`

**\*\*Importante:\*\***

Asegúrate de que la velocidad del Monitor Serie coincida con la velocidad configurada en tu código (`Serial.begin()`). Si usas 9600 en tu código, debes seleccionar 9600 en el Monitor Serie.

## Tipos de datos que puedes enviar

Con `Serial.print()` y `Serial.println()` puedes enviar diferentes tipos de datos:

```
Serial.println(42); // Número entero
Serial.println(3.14); // Número decimal
Serial.println('A'); // Un carácter
Serial.println("Hola, mundo!"); // Una cadena de texto
Serial.println(true); // Un valor booleano (imprime 1 para true, 0 para false)
```

## ## Ejemplo completo básico

```
void setup() {
 Serial.begin(9600); // Inicializa comunicación serial
 Serial.println("¡Arduino iniciado!");
}
void loop() {
 Serial.println("Arduino está funcionando");
 delay(1000); // Espera 1 segundo antes de repetir
}
```

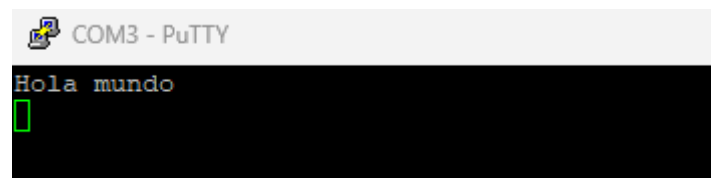
En este caso, como el texto solo queremos imprimirlo 1 sola vez, lo mejor sería ponerlo solo en el `setup()`:

```

P01_HolaMundo.ino
1 /*
2 programa 1: Hola mundo arduino
3 */
4
5 void setup() {
6
7 Serial.begin(9600);
8 Serial.println("Hola mundo");
9 }
10
11 void loop() {
12
13 }
14

```

Salida por consola:



```

COM3 - PuTTY
Hola mundo

```

## Puntos clave a recordar

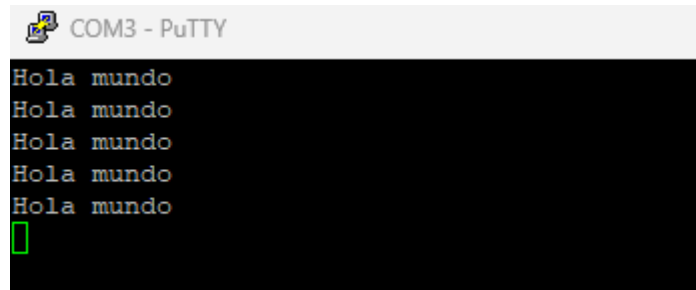
- **\*\*Siempre inicializa\*\*** Serial en `setup()` con `Serial.begin()`
- **\*\*Asegúrate\*\*** de que la velocidad del Monitor Serie coincida con tu código
- **\*\*Usa `println()`\*\*** cuando quieras cada dato en una línea nueva
- **\*\*Usa `print()`\*\*** cuando quieras controlar el formato exacto
- La comunicación serial es tu mejor amiga para depurar y entender qué hace tu código

En el 2º programa P02, se muestra un mensaje en bucle con un retardo de 1 segundo entre mensaje y mensaje:

```

P02_Mensaje_repetitivo.ino
1 ✓ /*
2 programa 2: Mensaje que se repiite cada 1 segundo
3 */
4
5 ✓ void setup() {
6
7 Serial.begin(9600); // Comunicación serial para visualizar resultado por consola
8
9 }
10
11 ✓ void loop() {
12 Serial.println("Hola mundo"); // mostrar texto por pantalla
13 delay(1000); // retardo de 1000 ms entre texto y texto
14 }
15

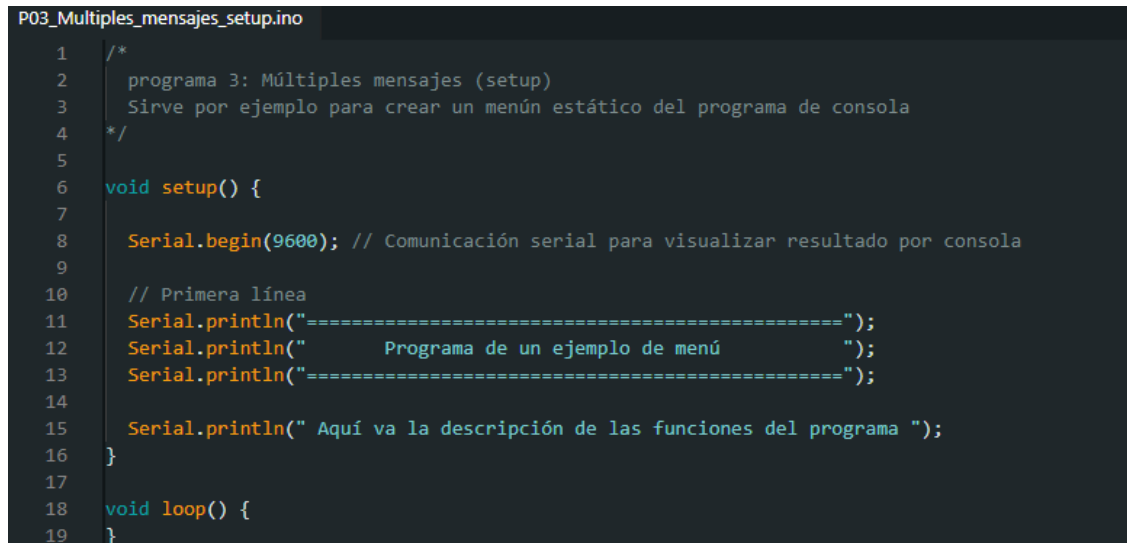
```



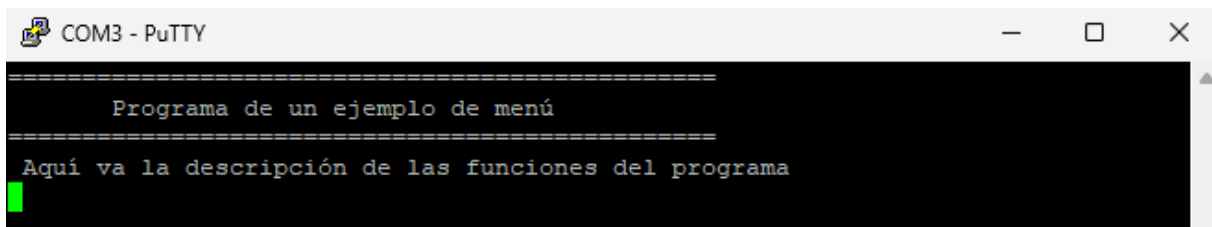
```
COM3 - PuTTY
Hola mundo
Hola mundo
Hola mundo
Hola mundo
Hola mundo
█
```

## Menú de consola

En los programas de consola es habitual que se muestre un menú para indicarle al usuario qué opciones tiene a la hora de interactuar con el programa. Como solo vamos a visualizar un ejemplo sencillo no funcional, lo podemos hacer perfectamente en el `setup()`:



```
P03_Multiples_mensajes_setup.ino
1 /*
2 programa 3: Múltiples mensajes (setup)
3 Sirve por ejemplo para crear un menú estático del programa de consola
4 */
5
6 void setup() {
7
8 Serial.begin(9600); // Comunicación serial para visualizar resultado por consola
9
10 // Primera línea
11 Serial.println("=====");
12 Serial.println(" Programa de un ejemplo de menú ");
13 Serial.println("=====");
14
15 Serial.println(" Aquí va la descripción de las funciones del programa ");
16 }
17
18 void loop() {
19 }
```



```
COM3 - PuTTY
=====
 Programa de un ejemplo de menú =====
Aquí va la descripción de las funciones del programa
█
```

## Diferencias entre print() y println()

Con print() se muestran mensajes sin salto de línea, si se pone otro print(), el 2º mensaje se muestra a continuación del primero, con println() se incluye un salto de línea al final del texto:

```
P04_Diferencias_entre_print_y_println.ino
1 /*
2 programa 4: Diferencias entre print() y println()
3 Sirve por ejemplo para crear un menú estático del programa de consola
4 */
5
6 void setup() {
7
8 Serial.begin(9600); // Comunicación serial para visualizar resultado por consola
9
10 Serial.print("Este es el inicio de un mensaje con print("") "); Serial.print("Este es el final");
11 Serial.println("");
12 Serial.println("Este es el comienzo de un mensaje con println("") ");Serial.println("Este es el final");
13 }
14
15 void loop() {
16 }
```

```
COM3 - PuTTY
Este es el inicio de un mensaje con print() Este es el final
Este es el comienzo de un mensaje con println()
Este es el final
█
```